Aula 22: Diagramas de Interação e Máquina de Estados

Fundamentação Comportamental baseada nos Capítulos 7, 8 e 9 (Gilleanes Guedes)

Prof. Reginaldo Pereira Fernandes
Técnico em Informática para Internet

Instituto Federal do Ceará – Campus Tauá

3 de novembro de 2026

Sumário da Aula

1. Diagrama de Sequência (Capítulo 7)
2. Diagrama de Comunicação (Capítulo 8)
3. Diagrama de Máquina de Estados (Capítulo 9)
4. Mapeamento Prático em Python
5. Atividade Prática: TauáDelivery Web
6. Síntese e Fechamento

Da Estrutura Estática à Interação Temporal

A Dinâmica dos Objetos na Execução:

- Na **Aula 20 e 21**, modelamos o **Diagrama de Classes**: os tipos, atributos, métodos e associações existentes no sistema.
- Mas como os objetos **conversam entre si** para realizar a funcionalidade de um Caso de Uso?
- Na **UML 2**, a colaboração entre instâncias é expressa pelos **Diagramas de Interação**.

Papel do Diagrama de Sequência:

- Enfatiza a **ordenação temporal** da troca de mensagens;
- Permite visualizar o tempo de vida de cada participante e a duração de suas execuções.

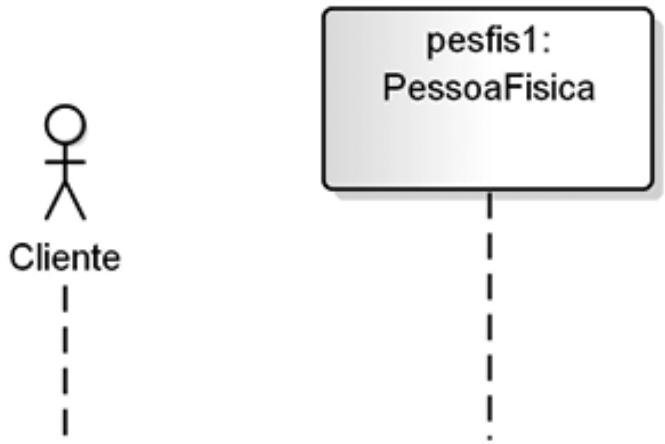
Perspectivas de Modelagem (Guedes)

- **Visão Temporal (Sequência):** Linha do tempo vertical; passos cronológicos evidentes de cima para baixo.
- **Visão Topológica (Comunicação):** Enfatiza os vínculos estruturais e conexões de rede entre objetos.

Anatomia do Diagrama de Sequência (Cap. 7)

Elementos Estruturais Básicos:

- **Linha de Vida (*Lifeline*):** Representa a existência do objeto no tempo. Cabeçalho no topo com retângulo (`objeto:Classe`).
- **Linha Tracejada Vertical:** Eixo temporal descendente (o tempo flui do topo para a base).
- **Barra de Ativação / Foco de Controle:** Retângulo estreito sobreposto indicando o período em que o objeto está executando uma operação ou aguardando resposta.
- **Atores:** Representados pelo símbolo clássico do *boneco-palito* no topo de sua linha de vida.

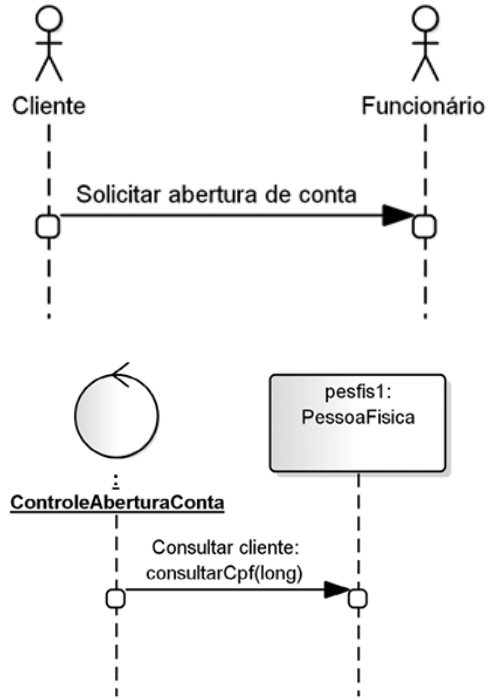


Lifeline de Ator (Fig. 7.1) e Objeto (Fig. 7.2 - Guedes)

Tipos de Mensagens e Estímulos na UML 2

Classificação Semântica das Mensagens:

- **Síncrona (▶):** Seta preenchida. O remetente **bloqueia a execução** até a resposta.
- **Assíncrona (→):** Seta aberta. O remetente despacha o sinal e prossegue imediatamente.
- **Resposta/Retorno (⇢):** Linha tracejada com retorno do valor calculado.
- **Autochamada (*Self-Call*):** Seta que retorna ao próprio objeto, gerando foco de controle empilhado.



Mensagens Síncronas (Fig. 7.3) e Autochamada (Fig. 7.4)

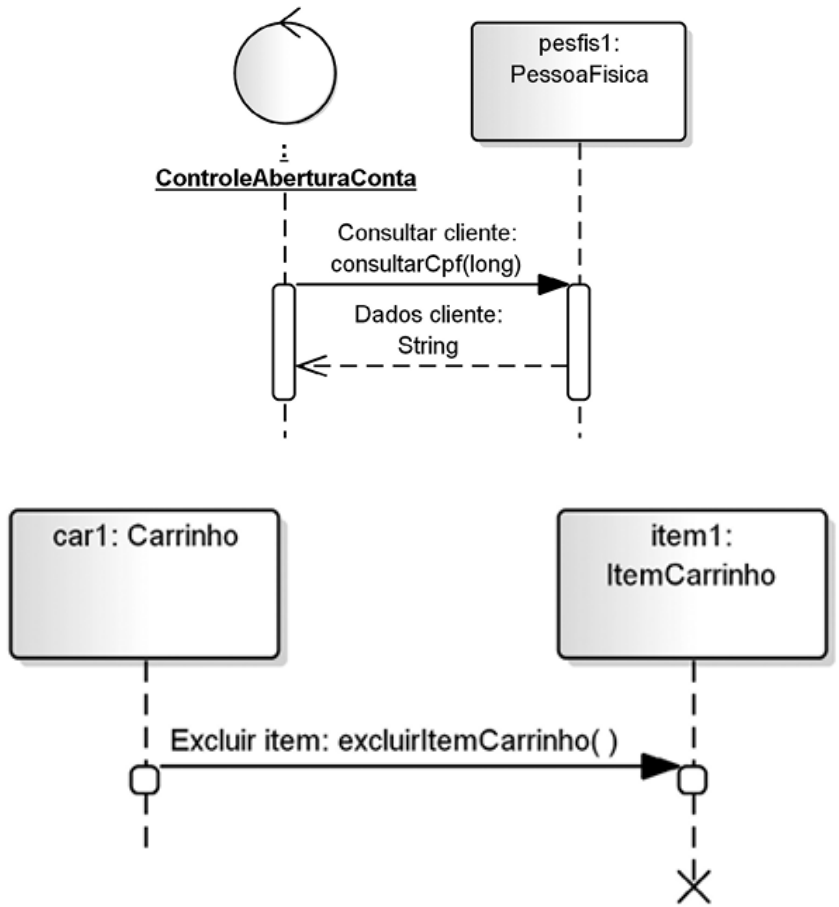
Criação e Destruição de Instâncias no Tempo

Mensagens Construtoras (*Create*):

- Mensagem com estereótipo «`create`» ou operação construtora (`new()`);
- A seta aponta **diretamente para a caixa de cabeçalho** do novo objeto, indicando que ele não existia no início do cenário.

Mensagens Destrutoras (*Destroy*):

- Mensagem com estereótipo «`destroy`» ou invocação de destruição explícita;
- A linha de vida do objeto é encerrada com um grande **marcador em cruz (×)**, indicando desalocação de memória.



Criação (Fig. 7.5) e Destruição com × (Fig. 7.7 - Guedes)

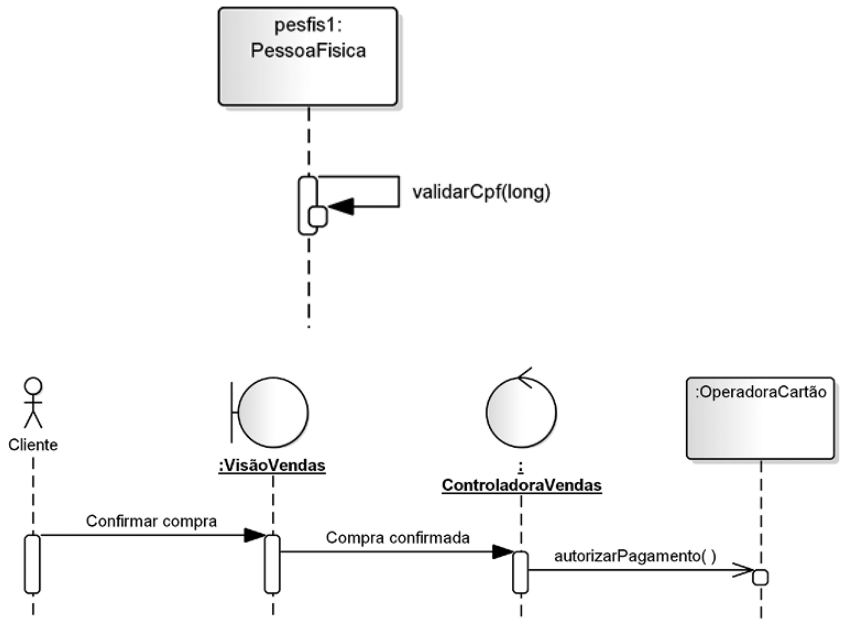
Fragmentos Combinados: Desvios Condicionais (`alt` e `opt`)

Avanço da UML 2: Fragmentos Combinados

- Caixas estruturadas com aba de operador no canto superior esquerdo e operandos delimitados.

Operadores de Desvio:

- **Operador** `alt` **(*Alternatives*):** Ramos mutuamente exclusivos (`if-else`) separados por linha tracejada com guardas `[condição]`.
- **Operador** `opt` **(*Option*):** Bloco condicional simples (`if` sem `else`); dispara apenas se a guarda for verdadeira.



Operador `alt` (Fig. 7.8) e Operador `opt` (Fig. 7.9)

Fragmentos Combinados: Repetição, Paralelismo e Ruptura

Operador `loop` (Iteração):

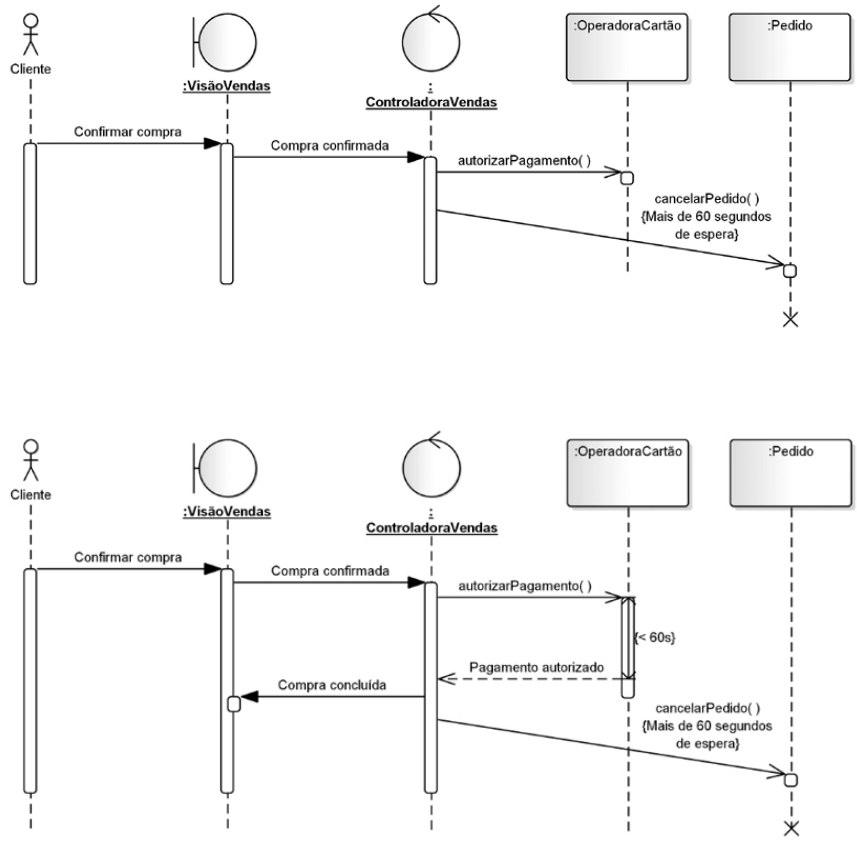
- Modela laços de repetição (`while`, `for`);
- Notação com intervalo de contagem e guarda: `loop(min, max)[guarda]`, ex.: `loop(1, n)[temMaisItens]`.

Operador `par` (Paralelismo):

- Define subcenários concorrentes executados em paralelo por diferentes threads.

Operador `break` (Ruptura / Exceção):

- Executado em situações anormais (exceções), abandonando o fluxo restante do fragmento pai.



Operador `loop` (Fig. 7.10) e Operador `par` (Fig. 7.11)

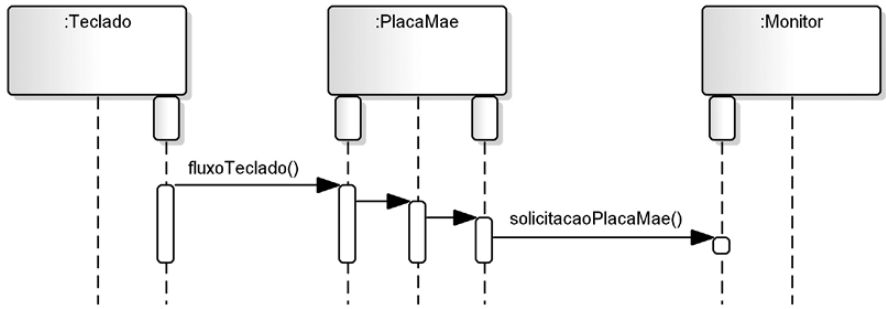
Recursos Avançados de Sequência (Fig. 7.13 e 7.15)

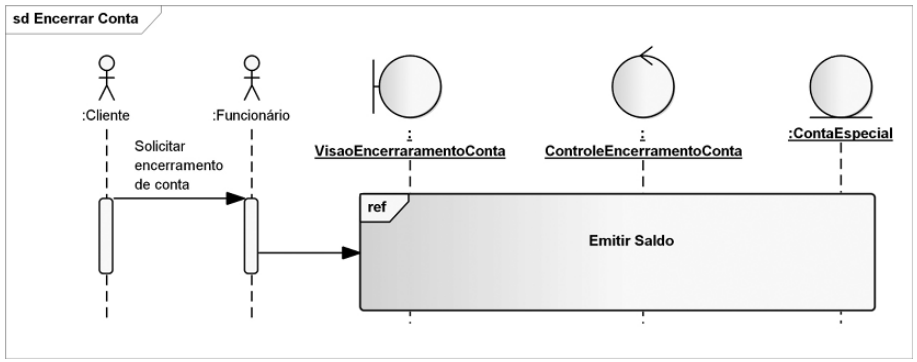
Invariante de Estado (*StateInvariant*):

- Símbolo de retângulo arredondado sobreposto à linha de vida contendo uma restrição ou estado formal (ex.: `{saldo >= 0}`);
- Deve ser obrigatoriamente verdadeiro para que o objeto possa receber mensagens naquele instante.

Uso de Interação (`ref`):

- Permite modularizar o diagrama, inserindo uma referência a outro Diagrama de Sequência já especificado (evita duplicação de diagramas longos).



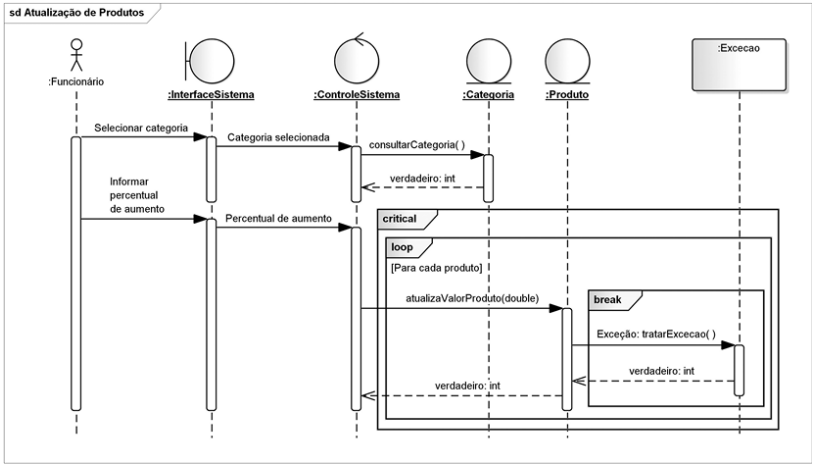


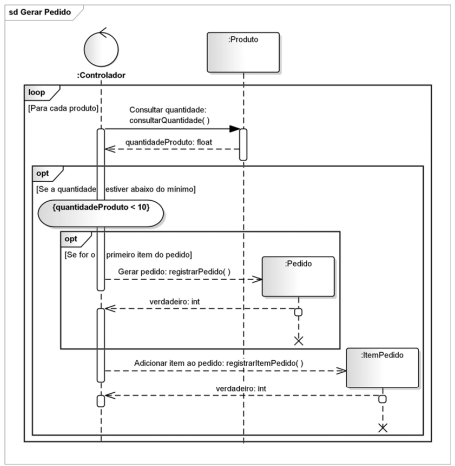
Invariante de Estado (Fig. 7.13) e Operador `ref` (Fig. 7.15)

Integração com Persistência: Padrões Repository e DAO

O Papel da Camada de Dados na Sequência:

- Na arquitetura em camadas (MVC/ECB), os objetos da camada de controle não executam SQL diretamente.
- O controller delega a persistência para uma instância **Repository** ou **DAO** (*Data Access Object*).
- A classe Repository monta a entidade a partir do banco e retorna instâncias prontas para as camadas superiores.
- O Diagrama de Sequência documenta com clareza o ciclo de leitura, validação e gravação no banco de dados.



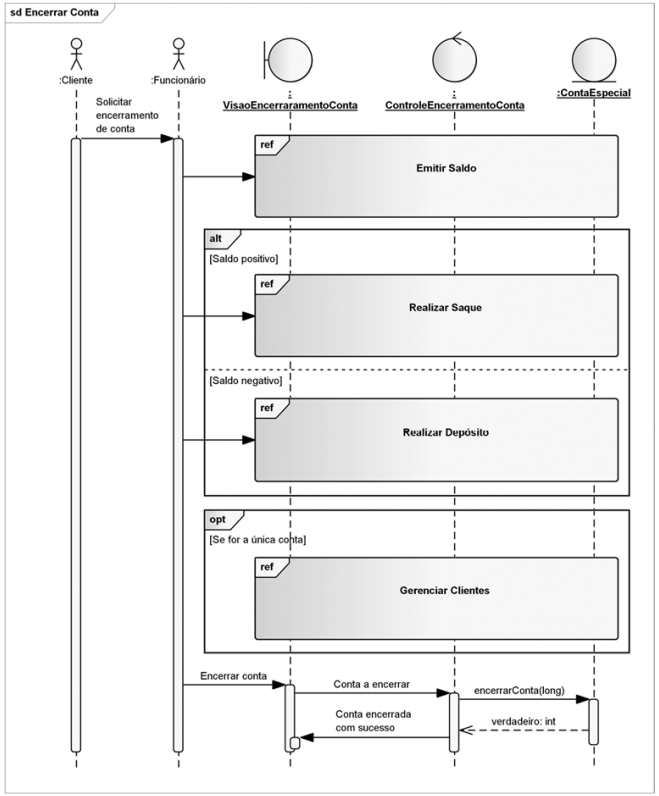


Chamadas a DAO e Repository no Diagrama de Sequência (Guedes)

Estudo de Caso Bancário: Realizar Saque na Sequência

Rastreamento do Fluxo:

1. O cliente aciona `efetuarSaque(num, senha, valor)` na tela (`«boundary»`);
2. O controle valida a senha da conta;
3. O fragmento `alt` avalia o saldo:
   - Se `saldo >= valor`: debita na conta, emite cédulas e grava log de movimento;
   - Senão (`else`): informa mensagem de saldo insuficiente ao cliente.



Processo de Saque Bancário no Diagrama de Sequência (Fig. 7.18 - Guedes)

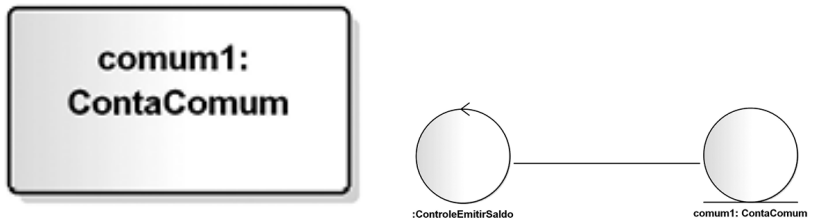
Introdução ao Diagrama de Comunicação (Cap. 8)

Origem e Propósito:

- Chamado de *Diagrama de Colaboração* na UML 1.x;
- É o segundo diagrama de interação da UML 2.
- Possui **equivalência semântica** com o Diagrama de Sequência, expressando as mesmas mensagens.

Mudança Fundamental de Perspectiva:

- Em vez de uma linha do tempo vertical, os objetos são dispostos como nós em uma **rede de conexões**;
- Enfatiza os **vínculos estruturais (*links*)** que possibilitam a comunicação entre as partes.



Lifeline (Fig. 8.1) e Vínculo com Mensagem (Fig. 8.2 - Guedes)

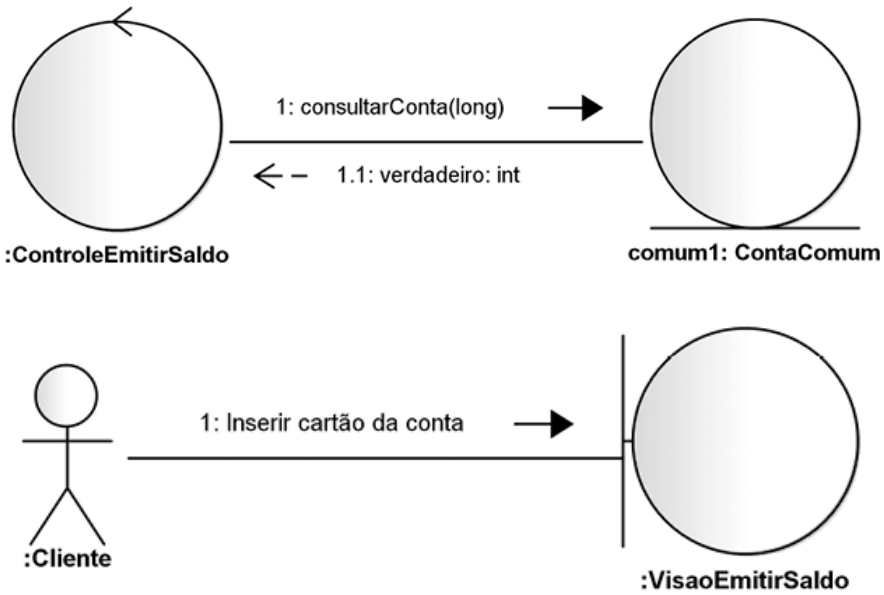
Numeração Hierárquica e Aninhamento de Mensagens

Ordenação Temporal via Numeração:

- A sequência temporal é governada por **numeração decimal aninhada**;
- Pequenas setas ao lado do vínculo indicam a direção.

Hierarquia de Aninhamento:

- `1: operacao()`: Mensagem inicial disparada;
- `1.1: subOpA()`: Chamada decorrente de 1;
- `1.2: subOpB()`: Segunda suboperação no escopo;
- `1.2.1: detalhe()`: Chamada aninhada de 1.2.



Mensagens Aninhadas (Fig. 8.3 e 8.4 - Guedes)

Autochamada, Condições de Guarda e Iterações

Autochamada (*Self-Call*):

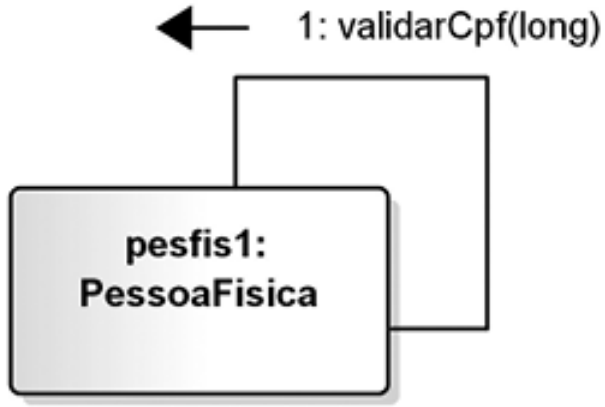
- Representada por um vínculo em arco ou laço que sai e retorna para o próprio objeto (Fig. 8.5).

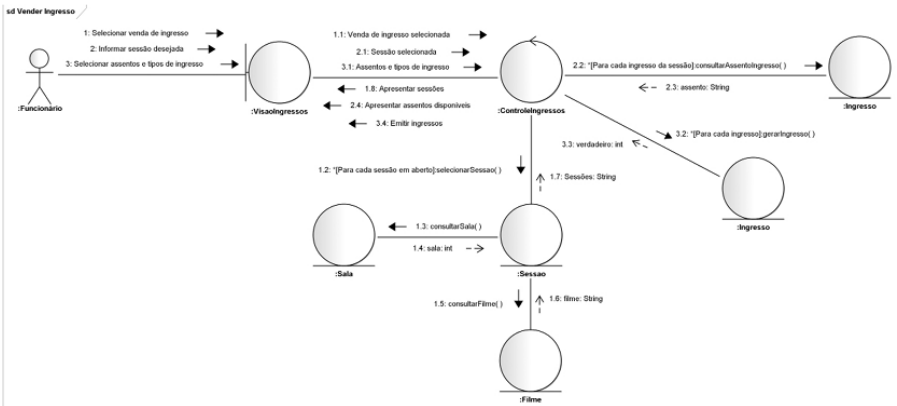
Condições de Guarda:

- Expressas entre colchetes antes do nome da operação, ex.: `1.1: [saldo >= valor] debitar()`.

Expressão de Iteração (Laços):

- Um asterisco (*) denota repetição sobre múltiplos elementos;
- Sintaxe completa: `* [i := 1..n]: calcularSubtotal()`.





Autochamada (Fig. 8.5) e Laço de Iteração (Fig. 8.8 - Guedes)

Estudo de Caso: Comunicação no Processo Emitir Saldo

Análise do Fluxo na Rede de Objetos:

- `1: emitirSaldo()`: Cliente solicita na interface `TelaEmitirSaldo`;
- `1.1: validaSenha()`: O controle aciona a conta bancária;
- `1.2: [senhaCorreta] obterSaldo()`: A conta calcula e retorna o saldo derivado;
- `1.3: exibirSaldo()`: A tela apresenta os dados ao cliente.

Vantagem Visual: Fica imediatamente claro quais instâncias dependem diretamente de quem para trocar dados.

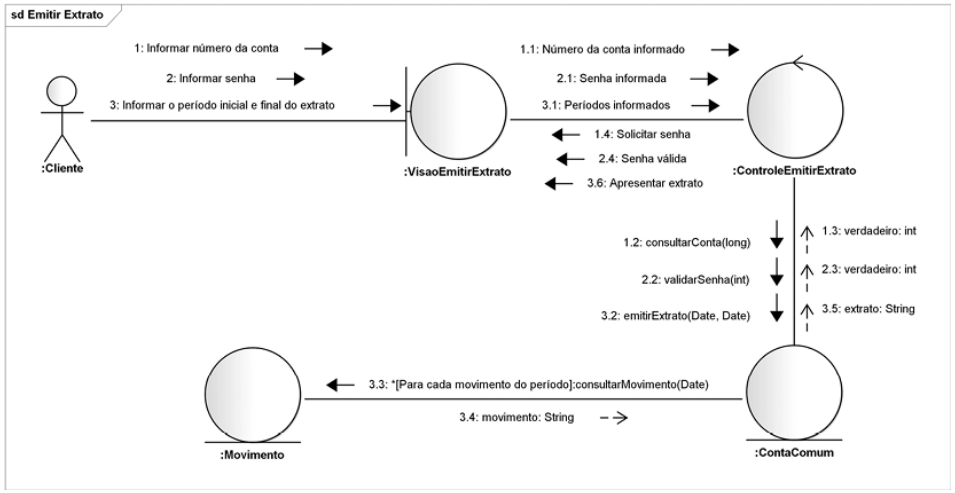


Diagrama de Comunicação do Processo Emitir Saldo (Fig. 8.7 - Guedes)

Análise Comparativa: Sequência vs. Comunicação

Característica	Diagrama de Sequência	Diagrama de Comunicação
Foco Principal	Ordenação temporal e cronologia de execução.	Topologia de rede e vínculos estruturais.
Representação do Tempo	Eixo vertical contínuo (de cima para baixo).	Numeração sequencial aninhada (`1.1`, `1.2`).
Foco de Controle / Ativação	Barras verticais detalhadas e explícitas.	Não representa visualmente o tempo de bloqueio.
Complexidade de Desvios	Fragmentos elegantes (`alt`, `loop`, `par`).	Guardas em linha e laços com `*`.
Melhor Cenário de Uso	Documentar cenários complexos de Casos de Uso.	Validar coesão e acoplamento entre classes.

Tabela: Quadro comparativo dos Diagramas de Interação (Capítulos 7 e 8 - Guedes).

Equivalência Semântica e Regras de Conversão

Como Converter Sequência em Comunicação:

1. Converta cada linha de vida em um objeto nó no diagrama de comunicação;
2. Crie um vínculo sólido entre pares de nós para cada mensagem trocada;
3. Atribua numeração decimal correspondente ao nível de aninhamento dos focos de controle da sequência;
4. Adicione a seta direcional indicando remetente $\rightarrow$ destinatário.

Regra de Ouro da UML 2

Ambos os diagramas expressam **exatamente o mesmo comportamento**. A escolha depende exclusivamente do aspecto que o engenheiro deseja destacar para a equipe de desenvolvimento.

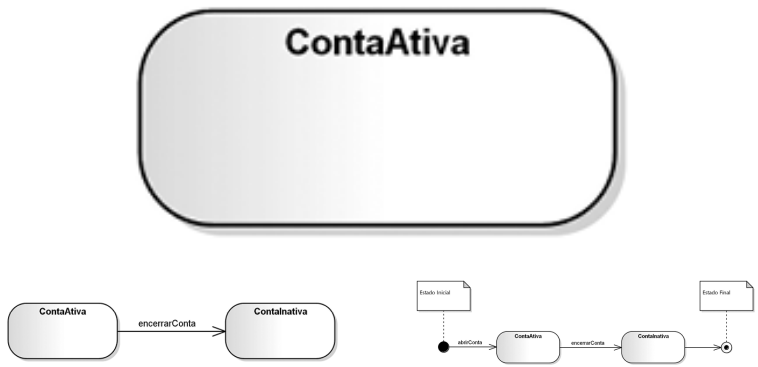
Introdução ao Diagrama de Máquina de Estados (Cap. 9)

Conceito de Estado e Ciclo de Vida:

- Condição na qual uma entidade **satisfaz determinada restrição**, executa ações ou aguarda eventos.
- Foca no **ciclo de vida de uma única classe reativa** (ex.: `Pedido`, `ContaComum`).

Elementos Fundamentais:

- **Estado Inicial (•):** Ponto de partida do ciclo;
- **Estado Simples:** Retângulo de cantos arredondados;
- **Estado Final (⊙):** Encerramento da vida da instância.



Estado Simples (Fig. 9.1), Inicial (9.2) e Final (9.3)

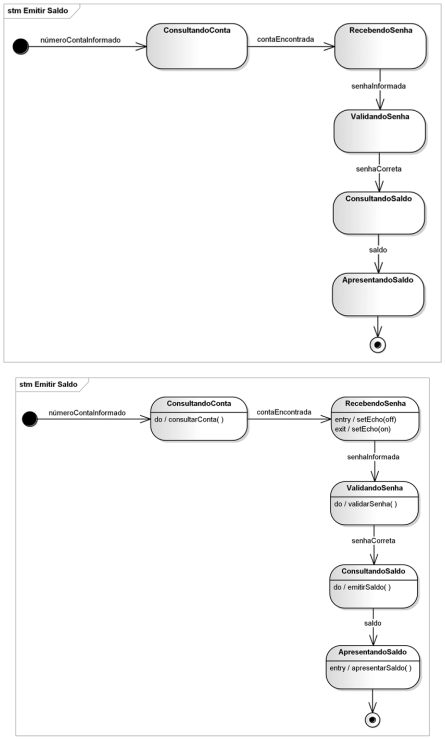
Transições Dirigidas por Eventos e Sintaxe Canônica

A Fórmula Canônica da Transição UML 2:

```
gatilho [guarda] / acao
```

Partes Constituintes:

- **Gatilho / Evento (*Trigger*):** Estímulo que dispara a transição (ex.: `pagtoAprovado`).
- **Condição de Guarda (*Guard*):** Expressão lógica entre colchetes. Se falsa, bloqueia o disparo (ex.: `[valor > 0]`).
- **Ação de Efeito (*Effect*):** Comportamento atômico instantâneo executado na transição (ex.: `/ emitirRecibo()`).



Sintaxe de Transição e Guardas (Fig. 9.4 e 9.5 - Guedes)

Atividades Internas do Estado (entry, do, exit)

Comportamentos Internos do Estado:

- `entry / acao`: Execução atômica instantânea **ao entrar** no estado.
- `do / atividade`: Execução contínua **enquanto permanece** no estado (interrompível por eventos).
- `exit / acao`: Execução atômica **ao deixar** o estado.

Exemplo Prático (`EM_ROTA`):

- `entry / iniciarTelemetriaGps()`
- `do / transmitirCoordenadas()`
- `exit / finalizarRastreamento()`



Atividades Internas (Fig. 9.6 - Guedes)

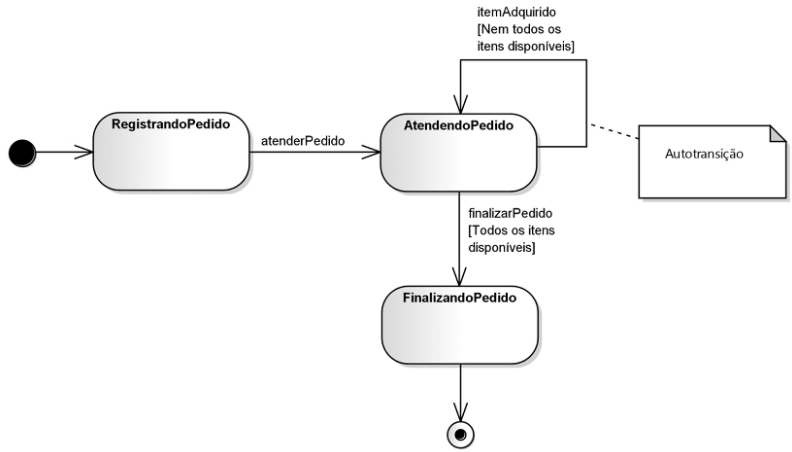
Transições Internas vs. Autotransições

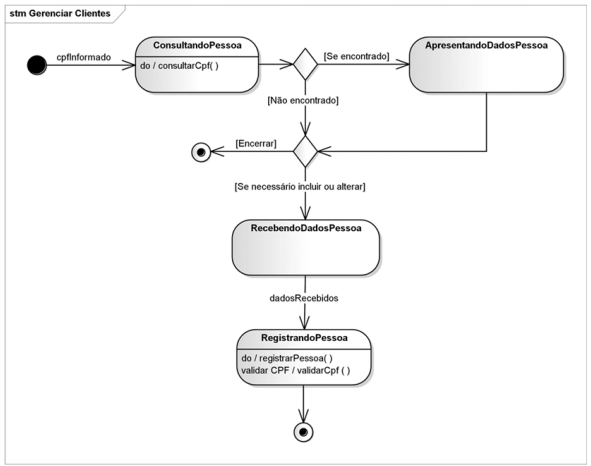
Transição Interna (Fig. 9.7):

- Responde a um evento alterando variáveis internas, mas **não provoca saída nem reentrada no estado**.
- **Ações** `entry` **e** `exit` **NÃO são disparadas**.

Autotransição (Fig. 9.8):

- Seta sai do contorno do estado e aponta de volta para o próprio estado.
- Provoca a saída formal e reentrada imediata.
- **Dispara** `exit` **e depois executa** `entry` **novamente**.





Transição Interna (Fig. 9.7) vs. Autotransição (Fig. 9.8 - Guedes)

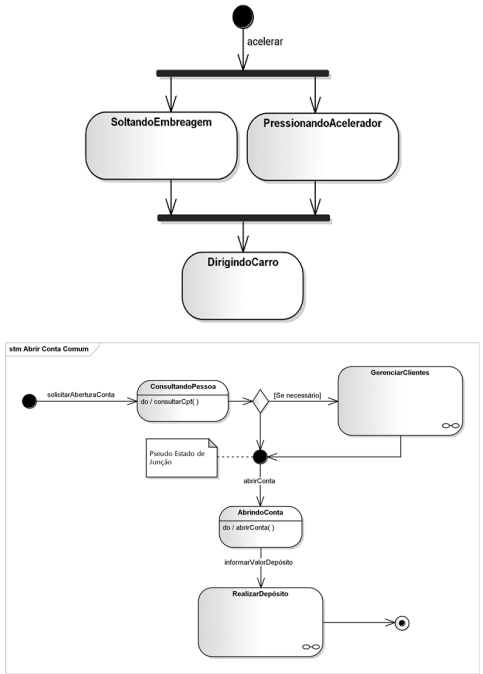
Pseudoestados de Decisão: Escolha (*Choice*) vs. Junção (*Junction*)

Escolha (*Choice* - ◊):

- Representado por um losango (Fig. 9.9);
- **Avaliação dinâmica em runtime**: as guardas são avaliadas quando o fluxo alcança o vértice;
- Suporta ramo padrão `[else]`.

Junção (*Junction* - •):

- Círculo sólido menor que o estado inicial (Fig. 9.16);
- **Avaliação estática preliminar**: todas as guardas são avaliadas antes do disparo da transição.



Escolha (Fig. 9.9) e Junção (Fig. 9.16 - Guedes)

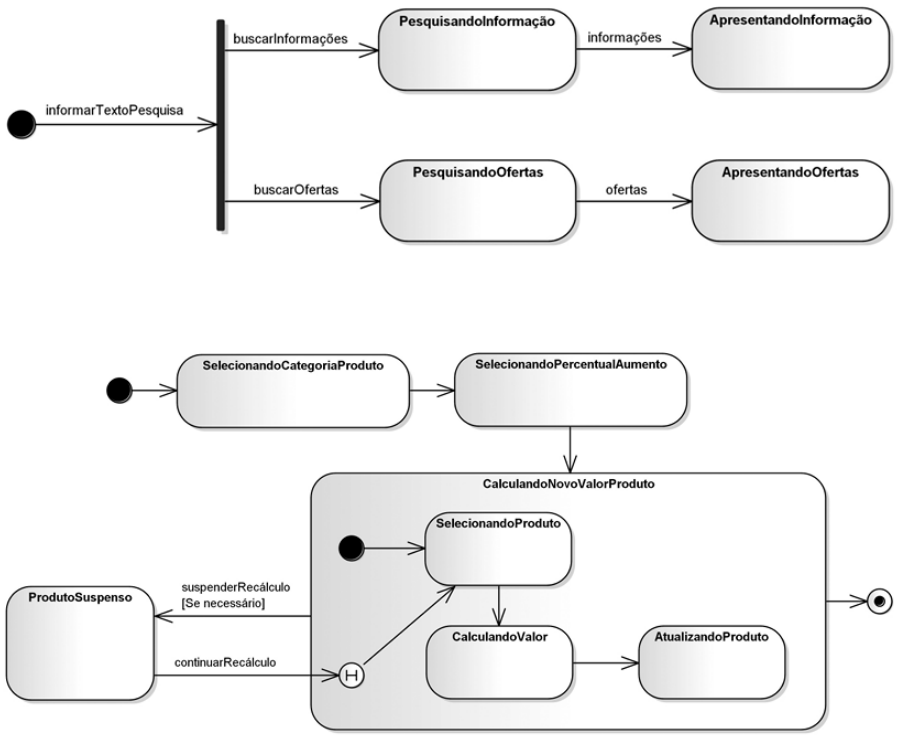
Bifurcação/União (*Fork/Join*) e Pseudoestado Histórico

Bifurcação / União (*Fork/Join*):

- Barra espessa horizontal ou vertical dividindo uma transição em duas ou sincronizando ramos paralelos.

Pseudoestado de Histórico (H e H^*):

- Permite a um estado composto **lembrar o último subestado ativo** antes de ter sido interrompido por uma transição externa.
- **Histórico Raso (H):** Restaura apenas o subestado do primeiro nível hierárquico.
- **Histórico Profundo (H^*):** Restaura o subestado em qualquer nível de profundidade recursiva.



Fork/Join (Fig. 9.10) e Pseudoestado de Histórico (Fig. 9.12)

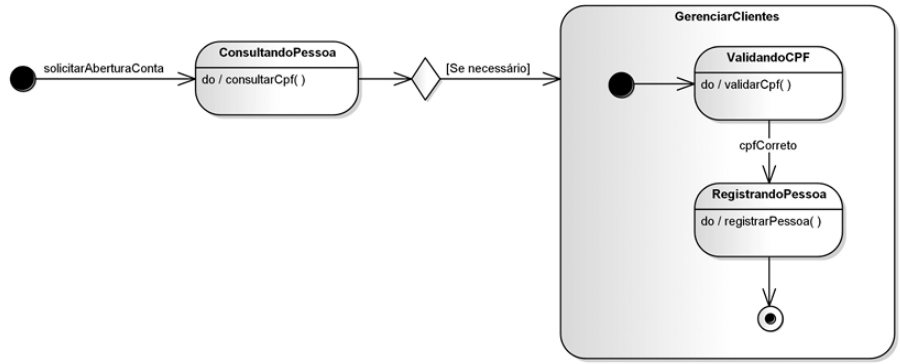
Estados Compostos e Regiões Ortogonais

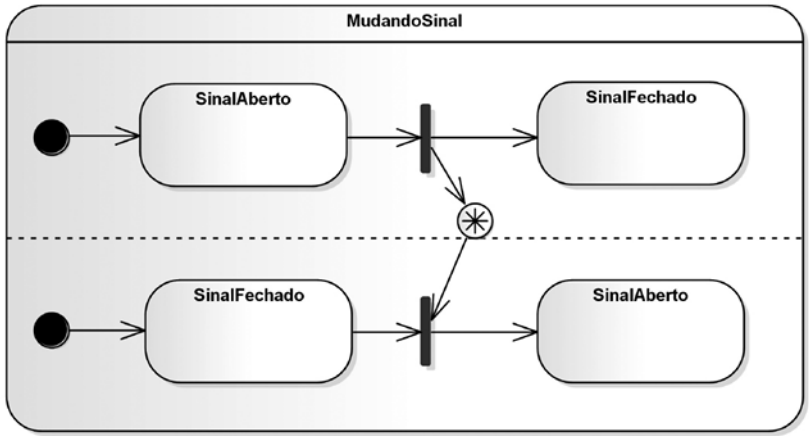
Estados Compostos (Subestados):

- Um estado simples pode conter uma máquina de estados inteira aninhada dentro de si (Fig. 9.11).
- Facilita a abstração de sistemas complexos.

Estados Ortogonais (Concorrência):

- O estado composto é dividido em duas ou mais regiões separadas por uma **linha pontilhada** (Fig. 9.14);
- O objeto está simultaneamente em um subestado de cada região ao mesmo tempo.





Estados Compostos (Fig. 9.11) e Ortogonais (Fig. 9.14)

Pseudoestados Especiais e Término

Estado de Submáquina:

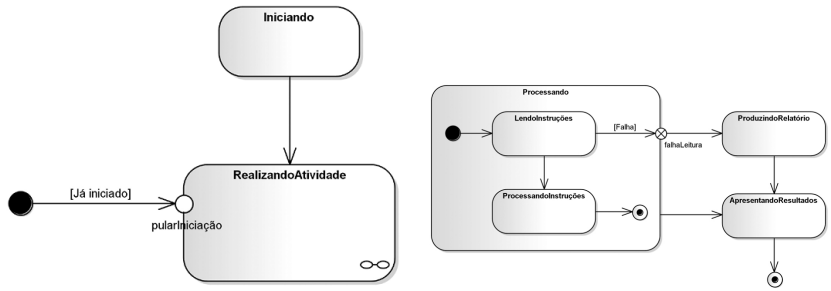
- Referencia uma máquina de estados externa reutilizável, indicada pelo símbolo de dois retângulos conectados no canto.

Ponto de Entrada e Ponto de Saída:

- Círculos na borda do estado composto permitindo saltar diretamente para subestados específicos.

Pseudoestado de Término ($\otimes$):

- Círculo com um **X** interno. Destrói o objeto imediatamente, sem disparar ações `exit`.



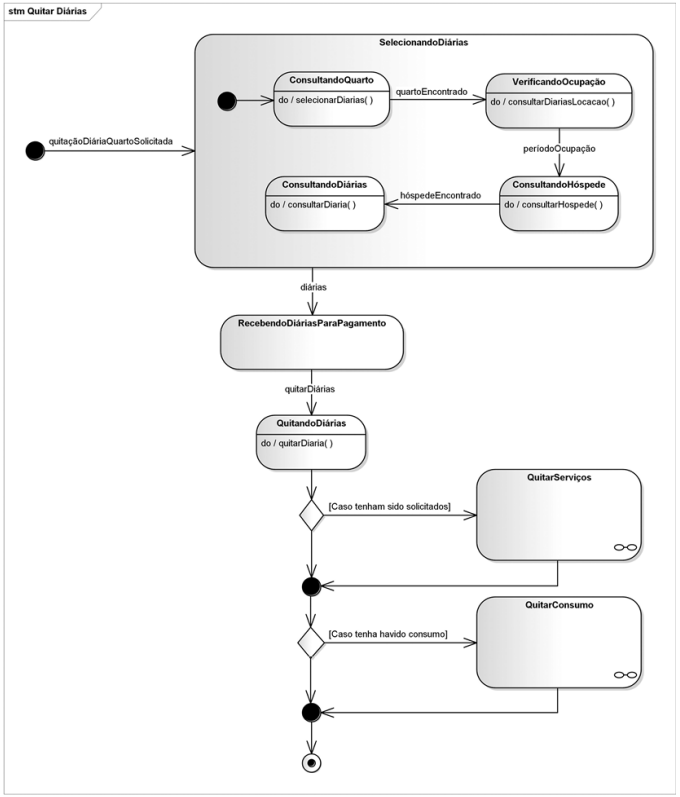
Pontos de Entrada/Saída (Fig. 9.17) e Término (Fig. 9.18)

Estudo de Caso: Ciclo de Vida da Conta Bancária

Ciclo de Vida de uma Conta Bancária:

- **Cadastrada:** Aguardando primeiro depósito;
- **Ativa:** Operando normalmente com saques e depósitos;
- **Bloqueada:** Em caso de erro de senha consecutivo;
- **Encerrada (⊙):** Encerramento formal após zerar o saldo.

Validação por Guardas: Transições como `encerrarConta [saldo == 0]` evitam fechar contas com pendências.



Máquina de Estados de Encerramento de Conta (Fig. 9.28 - Guedes)

Da Modelagem de Estados ao Código Python

```python
class StatusPedido(str, Enum):
    CRIADO = "CRIADO"
    AGUARDANDO_PAGAMENTO = "AGUARDANDO_PAGAMENTO"
    EM_PREPARO = "EM_PREPARO"
    PRONTO_PARA_COLETA = "PRONTO_PARA_COLETA"
    EM_ROTA_DE_ENTREGA = "EM_ROTA_DE_ENTREGA"
    ENTREGUE = "ENTREGUE"
    CANCELADO = "CANCELADO"

class PedidoStateMachine:
    def transitar_para(self, novo: StatusPedido, evento: str, guarda_ok: bool
        ):
        if not guarda_ok:
            raise TransicaoInvalidaException("Condicao de guarda falhou!")
        # 1. Executa acao exit / do estado de origem
        self._executar_exit(self.estado_atual)
        # 2. Transita estado
        self.estado_atual = novo
        # 3. Executa acao entry / do estado de destino
        self._executar_entry(self.estado_atual)
```

Executando o Rastreamento de Interação e Estados

O Script Didático:

- Arquivo: `exemplo_interacao_estados.py`;
- Demonstra a execução de checkout com persistência via Repository;
- Dispara mensagens síncronas, assíncronas e retornos;
- Simula a progressão da máquina de estados com tratamento de exceções de guarda.

Comando de Execução

```
$ python3 exemplo_interacao_estados.py
```

```
[Msg 1.2.1] PedidoService -> GatewayService:  debitar(PIX,
70.00)
[Msg 1.2.1.r] GatewayService -> PedidoService:
Transacao(APROVADA)
[alt -> Ramo Aprovado] Transicao:  AGUARDANDO_PAGAMENTO ->
EM_PREPARO
[entry / EM_PREPARO] Notificando painel KDS da cozinha.
```

Atividade Prática: TauáDelivery Web (Parte 2, 3 e 4)

Objetivo do Laboratório:

- Construir a modelagem comportamental dinâmica completa do caso de uso **UC03 - Finalizar Pedido e Processar Pagamento**.

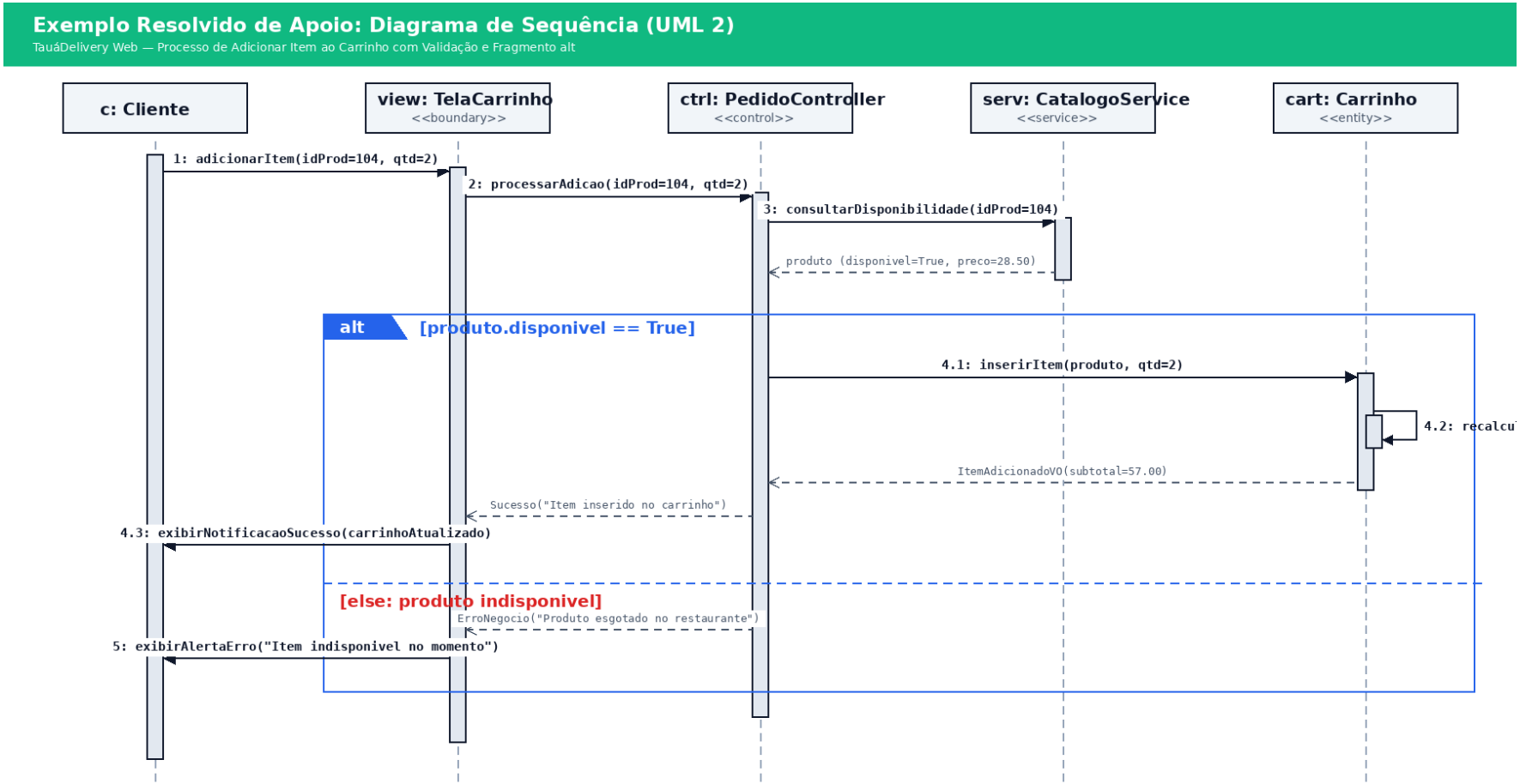
As 4 Etapas da Prática:

1. **Parte 1:** Questionário conceitual analítico;
2. **Parte 2:** Diagrama de Sequência completo com fragmento `alt` e repositório;
3. **Parte 3:** Diagrama de Comunicação com numeração decimal aninhada;
4. **Parte 4:** Diagrama de Máquina de Estados da entidade `Pedido`.

Entregáveis

Arquivo Markdown com diagramas em Mermaid.js ou PlantUML seguindo a rubrica de avaliação em `atividade_pratica.md`.

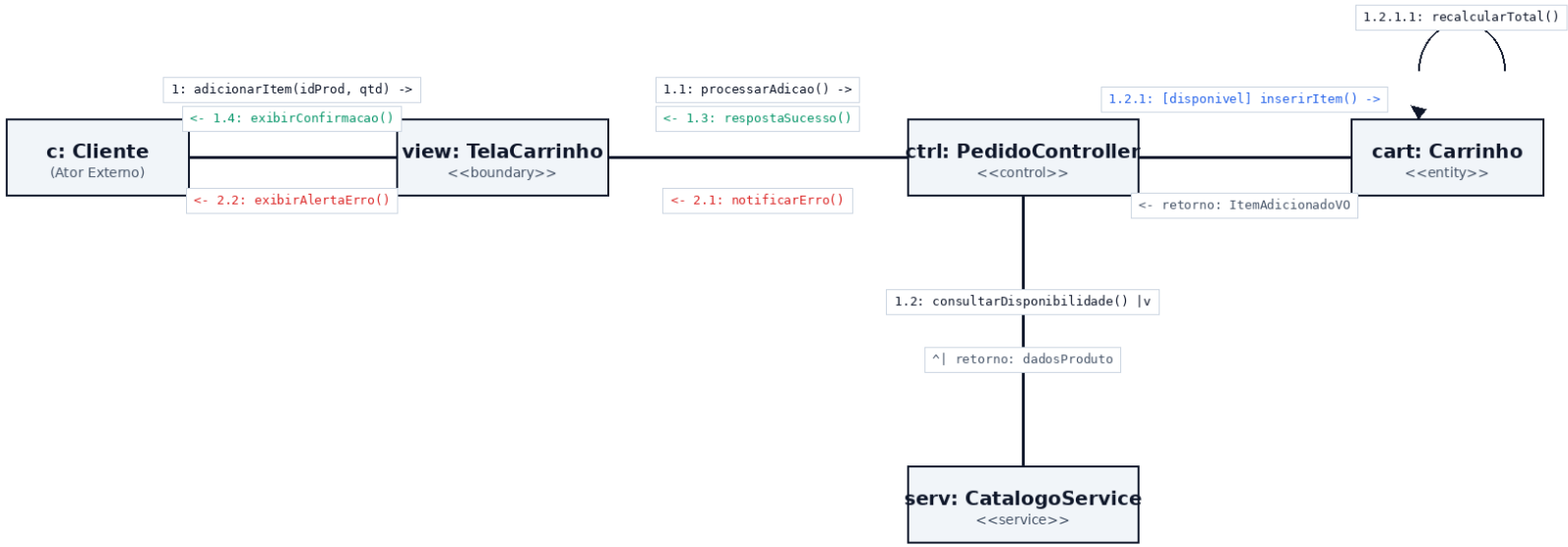
Exemplo Resolvido de Apoio 1: Diagrama de Sequência



Sequência de Adicionar Item ao Carrinho com Validação e Fragmento `alt`

Exemplo Resolvido de Apoio 2: Diagrama de Comunicação

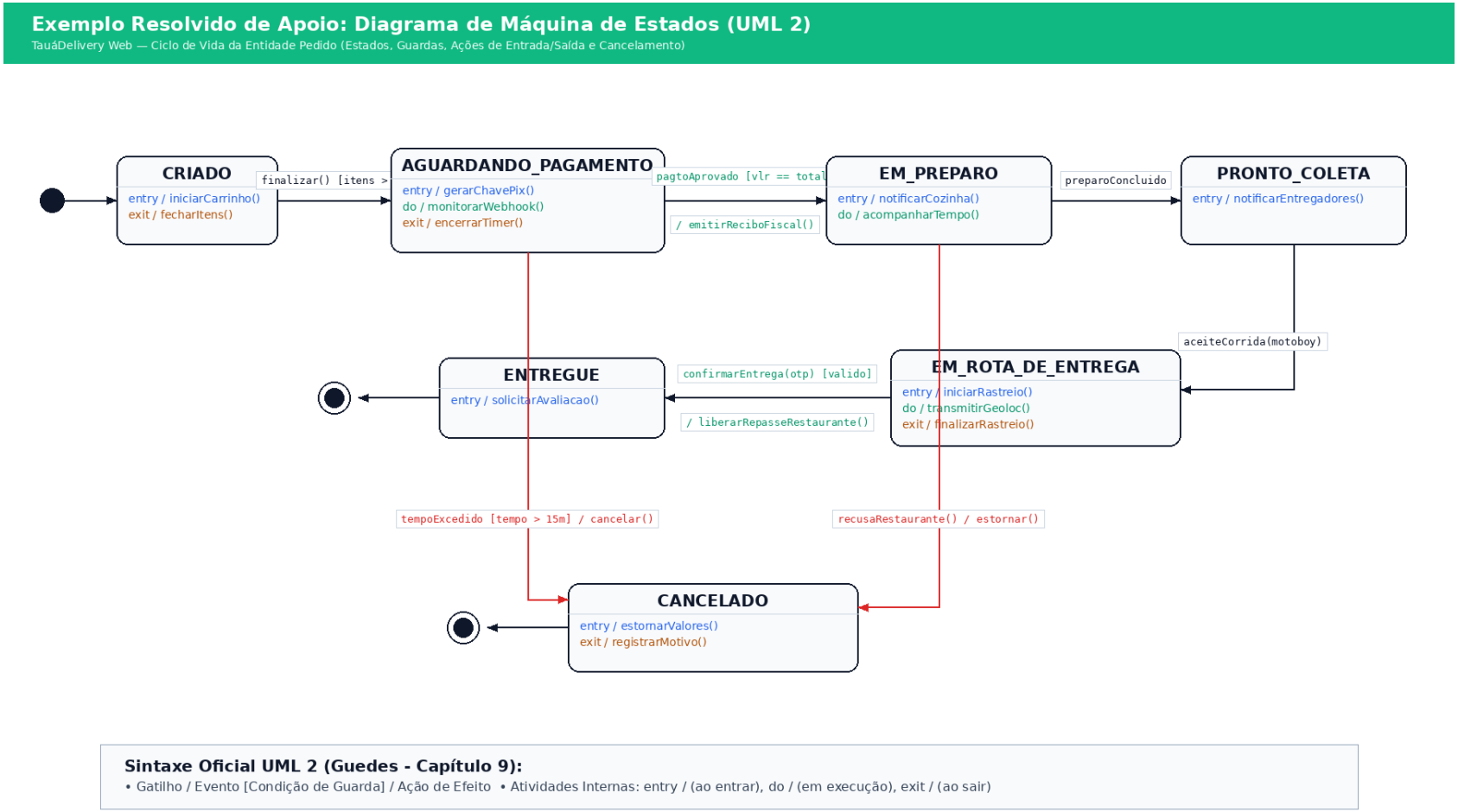
Exemplo Resolvido de Apoio: Diagrama de Comunicação (UML 2)
TauáDelivery Web — Equivalente Topológico com Vínculos Estruturais e Numeração Aninhada



Regra de Ouro (Guedes - Capítulo 8):
- O Diagrama de Comunicação expressa a mesma semântica comportamental do Diagrama de Sequência.
- Destaca os vínculos entre os objetos e adota numeração decimal aninhada (1, 1.1, 1.2, 1.2.1...) com setas indicando direção.

Diagrama de Comunicação correspondente com Vínculos e Numeração Aninhada

Exemplo Resolvido de Apoio 3: Máquina de Estados



Ciclo de Vida da Entidade Pedido com Guardas e Atividades Internas

Orientações de Entrega e Rubrica de Avaliação

Checklist de Modelagem Obrigatória:

- As 7 lifelines corretas na Sequência;
- Mensagem assíncrona na notificação;
- Fragmento `alt` com ramos aprovado e recusado;
- Numeração aninhada estrita na Comunicação;
- Todas as atividades `entry`, `do` e `exit` na Máquina de Estados.

Distribuição da Nota (10,0 pontos)

- **Parte 1 (Conceitual):** 2,5 pts
- **Parte 2 (Sequência):** 2,5 pts
- **Parte 3 (Comunicação):** 2,5 pts
- **Parte 4 (Máquina de Estados):** 2,5 pts

Síntese da Aula e Boas Práticas

Boas Práticas de Interação:

- Não sobrecarregue um único diagrama de sequência com dezenas de objetos; prefira dividir em subcenários com `ref`;
- Sempre explicite se a mensagem é síncrona ou assíncrona;
- Utilize o padrão Repository para abstrair consultas complexas.

Boas Práticas em Máquinas de Estados:

- Não use máquina de estados para modelar algoritmos ou fluxos procedurais (para isso, use o **Diagrama de Atividades**);
- Reserve a máquina de estados para entidades reativas com ciclo de vida significativo (ex.: Pedido, Sessão, Fatura).

Conclusão e Próximos Passos

Parabéns pela Conclusão da Aula 22!

Agora dominamos a estrutura estática (Classes), o fluxo procedimental (Atividades), a colaboração dinâmica (Sequência e Comunicação) e o ciclo de vida reativo (Máquina de Estados).

Próxima Aula (Aula 23)

Diagramas Arquiteturais da UML 2 (Capítulos 5, 6, 12 e 13 - Guedes):

- Diagrama de Pacotes e organização em camadas;
- Diagrama de Componentes e interfaces fornecidas/requeridas;
- Diagrama de Implantação e infraestrutura física/nuvem de servidores.